

MineWorld 观测平台端开发需求文档

版本: v1.0
日期: 2026-04-29
状态: 开发中
面向: 观测平台后端和前端开发者

目录

- 概述
- 设计原则
- 现有接口分析
- WebSocket 扩展
- HTTP REST API 扩展
- 数据库扩展
- Web 界面功能设计
- Agent World 集成
- 开发优先级
- 里程碑与验收标准

概述

文档目的

本文档是 MineWorld 观测平台 的独立开发需求文档，定义了观测平台后端的 WebSocket 扩展、HTTP REST API 新增、数据库扩展，以及 Web 界面功能设计。

核心定位

观测平台是核心差异化。类似 Neverland 但针对真实 Minecraft 世界，为多 Agent 编排提供实时监控、状态共享、操作审计等基础设施。

当前现状

- **WebSocket 接口**：4 种 Agent→服务端消息（agent:register、agent:status:update、agent:event、agent:world:snapshot）+ ping
- **HTTP API**：1 个管理接口（POST /api/admin/clear-data）
- **Web 界面**：初版上线，显示 Agent 在线数 + 接入地址
- **数据库**：PostgreSQL（agents、agent_events、agent_world_snapshots 三张表）

开发目标

扩展观测平台能力，支持：

- 6 种新增 WebSocket 消息类型
- 9 个新增 HTTP REST API 端点
- 数据库扩展（截图存储、轨迹数据等）
- 完整的 Web Dashboard（8 个功能模块）
- Agent World 身份集成

参考资料

- **Minecraft MindServer 架构**：<https://deepwiki.com/minecraft-bots/minecraft/6.1-mindserver-architecture> — MindServer 是 Minecraft 的多 Agent 协调中心，使用 Socket.IO 做进程管理和消息路由，Web UI 做实时监控。MineWorld 观测平台参考其看板和消息架构，但不做进程管理（Skill 端是独立脚本）
- **Minecraft Web UI**：<https://deepwiki.com/minecraft-bots/minecraft/6.2-web-ui-and-agent-control> — Agent 创建/设置/消息发送/状态展示的 Web 界面参考
- **观测平台 API 文档**：<https://bcebb916-73c2-4790-9efa-20b86711bb5c.dev.coze.site/api-docs.md> — 当前已实现接口的维护地址
- **Skill 端开发需求文档**：[./产品设计/minecraft-client_Skill端开发需求文档.md](#) — Skill 端 11 个新增脚本的完整设计，观测平台的每个接口必须与 Skill 脚本输出对齐

注意：Minecraft 的 MindServer 是进程管理器（管 Agent 的创建/销毁/重启），MineWorld 观测平台是被动观测者（只接收上报、不管理进程）。两者架构差异显著，只参考 Web UI 和消息路由思路。

设计原则

1. 向后兼容

所有新增接口与现有协议完全兼容，现有 Agent 无需修改即可继续工作。

2. 实时优先

WebSocket 作为核心通信方式，所有状态变更实时推送到 Observer。

3. 数据分层

- 内存层：实时状态（5 秒同步到数据库）
- 数据库层：持久化存储（事件、快照、截图）
- 对象存储：大文件（截图原图、地图瓦片）

4. 横向扩展

支持多实例部署，通过 PostgreSQL 和对象存储共享状态。

5. 与 MindServer 的差异化定位

Minecraft 的 MindServer 是**进程管理器**（创建/销毁/重启 Agent 进程），MineWorld 观测平台是**被动观测者**：

能力	MindServer	MineWorld
Agent 进程管理	✔ spawn/kill/restart	✘ 只接收上报
Agent 间消息路由	✔ Socket.IO 直接转发	✔ 广播给 Observer
Web UI	✔ Agent 创建/设置/消息	✔ 状态监控/截图/回放
身份体系	内置	Agent World 集成
截图系统	✘ 无	✔ 核心差异化
轨迹回放	✘ 无	✔ 核心差异化

核心差异：MineWorld 不管理 Agent 生命周期，专注于**可观测性**——让人类看到 Agent 在做什么。

现有接口分析

现有 WebSocket 消息

消息类型	方向	说明
agent:register	Agent→服务端	注册 Agent
agent:register:ack	服务端→Agent	注册确认
agent:status:update	Agent→服务端	更新状态
agent:event	Agent→服务端	上报事件
agent:world:snapshot	Agent→服务端	世界快照
ping	Agent→服务端	心跳
pong	服务端→Agent	心跳响应
observer:register	Observer→服务端	注册观测者
agents:list	服务端→Observer	Agent 列表
agent:registered	服务端→Observer	新注册通知
status:update	服务端→Observer	状态广播
event:new	服务端→Observer	事件广播
world:snapshot	服务端→Observer	快照广播
admin:data-cleared	服务端→Observer	数据清空通知

现有 HTTP API

端点	方法	说明
/api/admin/clear-data	POST	清空数据（scope: events/all）

当前限制

- 1.API 数量不足：仅有 1 个管理接口，无法满足前端查询需求
- 2.WebSocket 消息有限：缺少截图、建造进度、团队协作等消息类型
- 3.无截图存储：无法处理 Agent 上传的视觉信息
- 4.轨迹数据未存储：无法回放 Bot 移动轨迹
- 5.Web 功能不完整：仅有首页，缺乏详细监控模块

WebSocket 扩展

新增 Agent → 服务端消息

1. agent:vision - 截图上报

说明：Agent 上报视觉截图到观测平台

Agent 发送：

```
json
{
  "type": "agent:vision",
  "payload": {
    "agentId": "my-agent-001",
    "vision": {
      "captureId": "cap_abcdef",
      "imageData": "data:image/jpeg;base64,/9j/4AAQ...",
      "thumbnailData": "data:image/jpeg;base64,/9j/4AAQ...",
      "dimensions": { "width": 800, "height": 512 },
      "position": { "x": 100, "y": 64, "z": 200 },
      "facing": { "yaw": 45.5, "pitch": -10.2 },
      "description": "发现钻石矿",
      "scene": {
        "biome": "plains",
        "timeOfDay": "day",
        "weather": "clear",
        "dimension": "overworld"
      }
    },
    "timestamp": 1714406400000
  }
}
```

字段说明：

字段	类型	必填	说明
agentId	string	是	Agent ID
vision.captureId	string	是	截图唯一标识

字段	类型	必填	说明
vision.imageData	string	是	Base64 编码的图片数据（带 MIME 类型）
vision.thumbnailData	string	否	Base64 编码的缩略图
vision.dimensions	object	是	图片尺寸 { width, height }
vision.position	object	是	截图位置 { x, y, z }
vision.facing	object	否	朝向 { yaw, pitch }
vision.description	string	否	截图描述
vision.scene	object	否	场景信息
vision.timestamp	number	是	时间戳（毫秒）

服务端回复：

```
json
{
  "type": "vision:received",
  "payload": {
    "agentId": "my-agent-001",
    "captureId": "cap_abcdef",
    "success": true,
    "storageUrl": "https://mc-observer.coze.site/api/vision/cap_abcdef"
  }
}
```

服务端广播给 Observer：

```
{
  "type": "vision:new",
  "payload": {
    "agentId": "my-agent-001",
    "username": "MinerBot",
  }
}
```

```
json  "vision": {
      "captureId": "cap_abcdef",
      "thumbnailData": "data:image/jpeg;base64,/9j/4AAQ...",
      "dimensions": { "width": 800, "height": 512 },
      "position": { "x": 100, "y": 64, "z": 200 },
      "description": "发现钻石矿",
      "scene": {
        "biome": "plains",
        "timeOfDay": "day",
        "weather": "clear"
      },
      "timestamp": 1714406400000
    }
  }
}
```

实现要求：

- Base64 图片解码后存储到 S3/OSS
- 存储路径： `/vision/{agentId}/{captureId}.jpg`
- 缩略图生成（200x128，保持 800:512 宽高比）
- 数据库记录元数据到 `agent_vision` 表

2. agent:build:progress - 建造进度

说明：Agent 上报建造任务进度

Agent 发送：

```
{
  "type": "agent:build:progress",
  "payload": {
    "agentId": "my-agent-001",
    "build": {
      "buildId": "build_12345",
      "blueprintName": "simple_house",
      "status": "in_progress",
      "progress": 0.65,
      "currentLayer": 2,
      "totalLayers": 4,
      "blocksPlaced": 64,

```

```

json      "blocksTotal": 98,
          "materialsUsed": [
            { "material": "cobblestone", "used": 50, "remaining": 450 },
            { "material": "oak_planks", "used": 10, "remaining": 54 }
          ],
          "startedAt": 1714405620000,
          "estimatedCompletion": 1714406100000,
          "errors": []
        }
      }
    }
  }
}

```

字段说明：

字段	类型	必填	说明
agentId	string	是	Agent ID
build.buildId	string	是	建造任务 ID
build.blueprintName	string	是	蓝图名称
build.status	enum	是	in_progress / completed / failed / started / cancelled
build.progress	number	是	进度 (0.0 – 1.0)
build.currentLayer	number	否	当前层（多层建筑 时）
build.totalLayers	number	否	总层数
build.blocksPlaced	number	是	已放置方块数
build.blocksTotal	number	是	总方块数
build.materialsUsed	array	否	材料使用情况
build.startedAt	number	是	开始时间戳
build.estimatedCompletion	number	否	预计完成时间戳
build.errors	array	否	错误列表

服务端广播给 Observer:

```
json
{
  "type": "build:progress",
  "payload": {
    "agentId": "my-agent-001",
    "username": "MinerBot",
    "build": {
      "buildId": "build_12345",
      "blueprintName": "simple_house",
      "status": "in_progress",
      "progress": 0.65,
      "blocksPlaced": 64,
      "blocksTotal": 98,
      "timestamp": 1714406400000
    }
  }
}
```

实现要求:

- 进度更新存入内存（5 秒批量写入数据库）
- `agent_builds` 表记录建造任务状态
- 状态变更时广播给所有 Observer

3. agent:subscribe - 事件订阅

说明: Agent 告诉平台它想接收哪些游戏事件的推送通知

Agent 发送:

```
{
  "type": "agent:subscribe",
  "payload": {
    "agentId": "my-agent-001",
    "subscription": {
      "subscriptionId": "sub_12345",
      "events": ["entity_spawn", "player_death", "chat_received"],
      "filter": {
```

```
json      "entityTypes": ["zombie", "skeleton", "creeper"],
          "chatContains": null
        },
        "callbackUrl": null
      }
    }
  }
```

字段说明：

字段	类型	必填	说明
agentId	string	是	Agent ID
subscription.subscriptionId	string	是	订阅 ID
subscription.events	array	是	订阅的事件类型列表
subscription.filter	object	否	事件过滤器
subscription.callbackUrl	string	否	回调 URL（可选）

支持的 events：

- entity_spawn ： 实体生成
- entity_death ： 实体死亡
- player_death ： 玩家死亡
- chat_received ： 收到聊天
- inventory_changed ： 背包变化
- health_change ： 生命变化
- dimension_change ： 维度切换
- weather_change ： 天气变化

服务端回复：

```
{
  "type": "event:subscribed",
  "payload": {
    "agentId": "my-agent-001",
```

```
json  "subscription": {
      "subscriptionId": "sub_12345",
      "status": "active",
      "events": ["entity_spawn", "player_death", "chat_received"]
    }
  }
}
```

服务端推送订阅事件：

当订阅的事件发生时，服务端推送：

```
json
{
  "type": "event:subscribed",
  "payload": {
    "agentId": "my-agent-001",
    "subscriptionId": "sub_12345",
    "event": {
      "type": "entity_spawn",
      "entityType": "zombie",
      "position": { "x": 105, "y": 64, "z": 198 },
      "distance": 8.5,
      "timestamp": 1714406400000
    }
  }
}
```

Agent 取消订阅：

```
json
{
  "type": "agent:subscribe",
  "payload": {
    "agentId": "my-agent-001",
    "subscription": {
      "subscriptionId": "sub_12345",
      "action": "unsubscribe"
    }
  }
}
```

实现要求：

- 内存中维护订阅映射 `agentId → subscriptions[]`
- 事件发生时检查过滤条件并推送
- `agent_subscriptions` 表持久化订阅配置

4. agent:team:update - 团队状态

说明：Agent 上报团队/多 Bot 协作状态

Agent 发送：

```
{
  "type": "agent:team:update",
  "payload": {
    "agentId": "my-agent-001",
    "team": {
      "teamId": "team_001",
      "teamName": "MiningTeam",
      "action": "status_update",
      "leader": "my-agent-001",
      "members": [
        {
          "agentId": "my-agent-001",
          "username": "MinerBot",
          "role": "leader",
          "position": { "x": 100, "y": 64, "z": 200 },
          "status": "mining",
          "task": {
            "type": "mining_diamonds",
            "progress": 0.45,
            "targetPosition": { "x": 150, "y": 16, "z": 300 }
          }
        },
        {
          "agentId": "my-agent-002",
          "username": "GuardBot",
          "role": "member",
          "position": { "x": 95, "y": 64, "z": 195 },
          "status": "defending",
          "task": {
```

```

json      "type": "defend_area",
          "progress": 0.8,
          "targetPosition": { "x": 150, "y": 16, "z": 300 }
        }
      },
      "task": {
        "type": "coordinated_mining",
        "status": "in_progress",
        "progress": 0.45,
        "startedAt": 1714405620000,
        "estimatedCompletion": 1714408800000
      },
      "timestamp": 1714406400000
    }
  }
}

```

字段说明：

字段	类型	必填	说明
agentId	string	是	Agent ID
team.teamId	string	是	队伍 ID
team.teamName	string	是	队伍名称
team.action	enum	是	created / joined / left / status_update / disbanded
team.leader	string	是	队长 Agent ID
team.members	array	是	成员列表
team.members[].agentId	string	是	成员 ID
team.members[].username	string	是	成员用户名
team.members[].role	enum	是	leader / member
	object	否	当前位置

字段	类型	必填	说明
team.members[].position			
team.members[].status	string	否	当前状态
team.members[].task	object	否	当前任务
team.task	object	否	团队整体任务
team.timestamp	number	是	时间戳

服务端广播给 Observer：

```
json
{
  "type": "team:update",
  "payload": {
    "teamId": "team_001",
    "teamName": "MiningTeam",
    "action": "status_update",
    "leader": {
      "agentId": "my-agent-001",
      "username": "MinerBot"
    },
    "members": [
      { "agentId": "my-agent-001", "username": "MinerBot", "status": "mining" },
      { "agentId": "my-agent-002", "username": "GuardBot", "status": "defending" }
    ],
    "task": {
      "type": "coordinated_mining",
      "progress": 0.45
    },
    "timestamp": 1714406400000
  }
}
```

实现要求：

- `agent_teams` 表记录团队配置
- 内存中缓存团队状态
- 团队成员变更时广播更新

5. agent:chat - 聊天消息

说明：Agent 上报聊天消息，区分服务器聊天和 Agent 间通信

Agent 发送：

```
json
{
  "type": "agent:chat",
  "payload": {
    "agentId": "my-agent-001",
    "message": {
      "messageId": "msg_12345",
      "content": "发现钻石矿！位置 102, 55, 198",
      "channel": "public",
      "recipient": null,
      "sender": {
        "agentId": "my-agent-001",
        "username": "MinerBot",
        "type": "agent"
      },
      "mentionedAgents": [],
      "timestamp": 1714406400000
    }
  }
}
```

字段说明：

字段	类型	必填	说明
agentId	string	是	Agent ID
message.messageId	string	是	消息 ID
message.content	string	是	消息内容
message.channel	enum	是	public / whisper / team / system
message.recipient	string	否	私聊接收者 (channel=whisper 时)

字段	类型	必填	说明
message.sender	object	是	发送者信息
message.sender.agentId	string	是	发送者 Agent ID
message.sender.username	string	是	发送者用户名
message.sender.type	enum	是	agent / player
message.mentionedAgents	array	否	@提及的 Agent ID
message.timestamp	number	是	时间戳

channel 枚举:

- `public` : 公屏消息
- `whisper` : 私聊
- `team` : 队伍消息
- `system` : 系统消息

服务端广播给 Observer:

```
{
  "type": "chat:new",
  "payload": {
    "messageId": "msg_12345",
    "agentId": "my-agent-001",
    "username": "MinerBot",
    "message": {
      "content": "发现钻石矿! 位置 102, 55, 198",
      "channel": "public",
      "sender": {
        "type": "agent",
        "username": "MinerBot"
      },
      "mentionedAgents": [],
      "timestamp": 1714406400000
    }
  }
}
```

```
json }
}
```

实现要求：

- agent_messages 表存储聊天记录
- 每个 Agent 保留最近 100 条消息
- 支持按频道/时间查询

6. agent:trade - 村民交易

说明：Agent 上报村民交易事件

Agent 发送：

```
json
{
  "type": "agent:trade",
  "payload": {
    "agentId": "my-agent-001",
    "trade": {
      "tradeId": "trade_12345",
      "villagerId": 1234,
      "villagerProfession": "Librarian",
      "action": "trade_completed",
      "tradeIndex": 0,
      "count": 1,
      "input": ["emerald x5"],
      "output": "bookshelf x1",
      "position": { "x": 100, "y": 64, "z": 200 },
      "timestamp": 1714406400000
    }
  }
}
```

字段说明：

字段	类型	必填	说明
agentId	string	是	Agent ID

字段	类型	必填	说明
trade.tradeId	string	是	交易唯一标识
trade.villagerId	number	是	村民实体 ID
trade.villagerProfession	string	否	村民职业
trade.action	enum	是	trade_opened / trade_completed / trade_failed
trade.tradeIndex	number	是	交易槽位索引
trade.count	number	否	交易次数
trade.input	array	否	交易输入物品
trade.output	string	否	交易输出物品
trade.position	object	否	交易位置
trade.timestamp	number	是	时间戳

服务端广播给 Observer:

```
json
{
  "type": "trade:event",
  "payload": {
    "agentId": "my-agent-001",
    "username": "MinerBot",
    "trade": {
      "tradeId": "trade_12345",
      "action": "trade_completed",
      "villagerProfession": "Librarian",
      "input": ["emerald x5"],
      "output": "bookshelf x1",
      "timestamp": 1714406400000
    }
  }
}
```

实现要求：

- 交易事件通过现有 `agent_events` 表存储（`event_type` 为 `trade_opened`/`trade_completed`/`trade_failed`）
- 不需要单独建表，复用现有事件系统
- 广播给所有 Observer

HTTP REST API 扩展

通用说明

认证方式：Bearer Token

```
Authorization: Bearer <API_KEY>
```

通用响应格式：

```
json
{
  "success": true,
  "data": { ... },
  "meta": {
    "timestamp": "ISO8601",
    "requestId": "req_12345"
  }
}
```

错误响应格式：

```
json
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "错误描述"
  }
}
```

1. GET /api/agents

说明：获取所有 Agent 列表

请求参数：

参数	类型	必需	说明
status	enum	否	online, offline, all (默认 all)
server	string	否	服务器地址过滤
page	number	否	页码 (默认 1)
limit	number	否	每页数量 (默认 20, 最大 100)

响应示例：

```
{
  "success": true,
  "data": {
    "agents": [
      {
        "id": "my-agent-001",
        "username": "MinerBot",
        "serverHost": "mc.example.com",
        "serverPort": 25565,
        "connected": true,
        "position": { "x": 100, "y": 64, "z": 200 },
        "health": 20,
        "dimension": "overworld",
        "lastSeenAt": "2024-01-15T10:30:00.000Z",
        "createdAt": "2024-01-01T00:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 2,
      "totalPages": 1
    }
  }
}
```

```
json    }
      },
      "meta": {
        "timestamp": "2024-01-15T10:30:00.000Z",
        "requestId": "req_abc123"
      }
    }
  }
```

2. GET /api/agents/{id}

说明：获取 Agent 详情

响应示例：

```
{
  "success": true,
  "data": {
    "id": "my-agent-001",
    "username": "MinerBot",
    "serverHost": "mc.example.com",
    "serverPort": 25565,
    "connected": true,
    "status": {
      "position": { "x": 100, "y": 64, "z": 200 },
      "health": 20,
      "maxHealth": 20,
      "food": 18,
      "saturation": 5.0,
      "gamemode": "survival",
      "dimension": "overworld",
      "yaw": 180,
      "pitch": 0,
      "isOnGround": true,
      "isSprinting": false,
      "isSneaking": false
    },
    "inventory": [
      { "slot": 0, "name": "diamond_pickaxe", "displayName": "钻石镐", "count": 1 },
      { "slot": 1, "name": "cobblestone", "displayName": "圆石", "count": 64 }
    ],
    "equipment": {
```

```
json    "head": null,
        "chest": null,
        "legs": null,
        "feet": null,
        "mainhand": { "slot": 0, "name": "diamond_pickaxe", "displayName": "钻石镐",
"count": 1 },
        "offhand": null
    },
    "stats": {
        "totalEvents": 156,
        "blocksPlaced": 234,
        "blocksBroken": 567,
        "distanceTraveled": 12500,
        "itemsCollected": 890
    },
    "lastSeenAt": "2024-01-15T10:30:00.000Z",
    "createdAt": "2024-01-01T00:00:00.000Z"
},
"meta": {
    "timestamp": "2024-01-15T10:30:00.000Z",
    "requestId": "req_abc124"
}
}
```

3. GET /api/agents/{id}/events

说明：获取 Agent 事件历史

请求参数：

参数	类型	必需	说明
type	string	否	事件类型过滤
startTime	ISO8601	否	开始时间
endTime	ISO8601	否	结束时间
page	number	否	页码（默认 1）
limit	number	否	每页数量（默认 50，最大 200）

响应示例：

json

```
{
  "success": true,
  "data": {
    "events": [
      {
        "id": 1,
        "agentId": "my-agent-001",
        "type": "item_crafted",
        "description": "合成了 1 个钻石镐",
        "data": { "item": "diamond_pickaxe", "count": 1, "location": "inventory" },
        "createdAt": "2024-01-15T10:30:00.000Z"
      },
      {
        "id": 2,
        "agentId": "my-agent-001",
        "type": "block_broken",
        "description": "破坏了钻石矿石",
        "data": { "blockType": "diamond_ore", "position": { "x": 50, "y": 12, "z": -30 } },
        "createdAt": "2024-01-15T10:28:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 50,
      "total": 156,
      "totalPages": 4
    }
  },
  "meta": {
    "timestamp": "2024-01-15T10:30:00.000Z",
    "requestId": "req_abc125"
  }
}
```

4. GET /api/agents/{id}/snapshots

说明：获取 Agent 快照历史

请求参数：

参数	类型	必需	说明
page	number	否	页码（默认 1）
limit	number	否	每页数量（默认 20）

响应示例：

```
json
{
  "success": true,
  "data": {
    "snapshots": [
      {
        "id": 30,
        "agentId": "my-agent-001",
        "snapshot": {
          "blocks": [
            { "position": { "x": 100, "y": 63, "z": 200 }, "type": "grass_block", "name": "草方块" }
          ],
          "entities": [
            { "id": 1, "type": "pig", "name": "Pig", "position": { "x": 105, "y": 64, "z": 203 }, "distance": 7 }
          ]
        },
        "createdAt": "2024-01-15T10:30:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 30,
      "totalPages": 2
    }
  },
  "meta": {
    "timestamp": "2024-01-15T10:30:00.000Z",
    "requestId": "req_abc126"
  }
}
```

5. GET /api/agents/{id}/vision

说明：获取 Agent 截图历史

请求参数：

参数	类型	必需	说明
page	number	否	页码（默认 1）
limit	number	否	每页数量（默认 20）

响应示例：

```
{
  "success": true,
  "data": {
    "visions": [
      {
        "captureId": "cap_abcdef",
        "dimensions": { "width": 640, "height": 480 },
        "position": { "x": 100, "y": 64, "z": 200 },
        "facing": { "yaw": 45.5, "pitch": -10.2 },
        "description": "发现钻石矿",
        "scene": {
          "biome": "plains",
          "timeOfDay": "day",
          "weather": "clear"
        },
        "thumbnailUrl": "https://mc-observer.coze.site/api/vision/cap_abcdef/thumb",
        "imageUrl": "https://mc-observer.coze.site/api/vision/cap_abcdef",
        "createdAt": "2024-01-15T10:30:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 45,
      "totalPages": 3
    }
  },
  "meta": {
    "timestamp": "2024-01-15T10:30:00.000Z",
    "requestId": "req_abc127"
  }
}
```

```
json }  
}
```

6. GET /api/agents/{id}/trajectory

说明：获取 Agent 移动轨迹

请求参数：

参数	类型	必需	说明
startTime	ISO8601	否	开始时间（默认 24 小时前）
endTime	ISO8601	否	结束时间（默认现在）
resolution	enum	否	high, medium, low（默认 medium）

响应示例：

```
{  
  "success": true,  
  "data": {  
    "agentId": "my-agent-001",  
    "trajectory": {  
      "startPosition": { "x": 50, "y": 64, "z": 100 },  
      "endPosition": { "x": 150, "y": 55, "z": 200 },  
      "totalDistance": 215.3,  
      "points": [  
        { "x": 50, "y": 64, "z": 100, "t": 0, "timestamp": 1714405500000 },  
        { "x": 60, "y": 64, "z": 110, "t": 10, "timestamp": 1714405510000 },  
        { "x": 70, "y": 64, "z": 120, "t": 20, "timestamp": 1714405520000 },  
        { "x": 80, "y": 63, "z": 130, "t": 30, "timestamp": 1714405530000 },  
        { "x": 90, "y": 62, "z": 140, "t": 40, "timestamp": 1714405540000 },  
        { "x": 100, "y": 60, "z": 150, "t": 50, "timestamp": 1714405550000 },  
        { "x": 110, "y": 58, "z": 160, "t": 60, "timestamp": 1714405560000 },  
        { "x": 120, "y": 56, "z": 170, "t": 70, "timestamp": 1714405570000 },  
        { "x": 130, "y": 55, "z": 180, "t": 80, "timestamp": 1714405580000 },  
        { "x": 140, "y": 55, "z": 190, "t": 90, "timestamp": 1714405590000 },  
      ]  
    }  
  }  
}
```

```
json      { "x": 150, "y": 55, "z": 200, "t": 100, "timestamp": 1714405600000 }
],
"events": [
  { "type": "block_broken", "position": { "x": 102, "y": 55, "z": 198 }, "t": 75 },
  { "type": "item_picked_up", "position": { "x": 102, "y": 55, "z": 198 }, "t": 76 }
],
"bounds": {
  "minX": 50, "maxX": 150,
  "minY": 55, "maxY": 64,
  "minZ": 100, "maxZ": 200
}
},
"timeRange": {
  "start": "2024-01-15T10:20:00.000Z",
  "end": "2024-01-15T10:30:00.000Z"
}
},
"meta": {
  "timestamp": "2024-01-15T10:30:00.000Z",
  "requestId": "req_abc128"
}
}
```

实现要求：

- 从 `agent_status_updates` 表中提取位置历史
- 根据 `resolution` 参数采样：
 - `high` : 所有点
 - `medium` : 每 10 个点取 1 个
 - `low` : 每 100 个点取 1 个
- 关联事件数据标注轨迹关键点

7. GET /api/events

说明：获取全局事件流

请求参数：

参数	类型	必需	说明
agentId	string	否	Agent 过滤

参数	类型	必需	说明
type	string	否	事件类型
startTime	ISO8601	否	开始时间
endTime	ISO8601	否	结束时间
page	number	否	页码（默认 1）
limit	number	否	每页数量（默认 50）

响应示例：

```
{
  "success": true,
  "data": {
    "events": [
      {
        "id": 156,
        "agentId": "my-agent-001",
        "username": "MinerBot",
        "type": "item_crafted",
        "description": "合成了 1 个钻石镐",
        "data": { "item": "diamond_pickaxe", "count": 1 },
        "createdAt": "2024-01-15T10:30:00.000Z"
      },
      {
        "id": 155,
        "agentId": "my-agent-002",
        "username": "GuardBot",
        "type": "attacked",
        "description": "攻击了僵尸",
        "data": { "target": "zombie", "damage": 9 },
        "createdAt": "2024-01-15T10:29:30.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 50,
      "total": 12500,
      "totalPages": 250
    }
  }
},
```

```
json  "meta": {
      "timestamp": "2024-01-15T10:30:00.000Z",
      "requestId": "req_abc129"
    }
  }
```

8. GET /api/stats

说明：获取全局统计信息

响应示例：

```
{
  "success": true,
  "data": {
    "platform": {
      "totalAgents": 1250,
      "onlineAgents": 345,
      "totalServers": 28,
      "onlineServers": 15
    },
    "bots": {
      "totalBots": 567,
      "onlineBots": 345,
      "offlineBots": 222,
      "totalEvents": 1250000
    },
    "servers": {
      "totalServers": 28,
      "onlineServers": 15,
      "totalPlayers": 89
    },
    "events": {
      "totalEvents": 1250000,
      "eventsToday": 15000,
      "topEventTypes": [
        { "type": "moved", "count": 450000 },
        { "type": "block_broken", "count": 125000 },
        { "type": "item_picked_up", "count": 98000 }
      ]
    }
  },
}
```

```
json  "vision": {
      "totalCaptures": 4500,
      "capturesToday": 120
    },
    "achievements": {
      "totalAchieved": 4500,
      "todayAchieved": 45
    },
    "timeRange": {
      "oldestEvent": "2024-01-01T00:00:00.000Z",
      "newestEvent": "2024-01-15T10:30:00.000Z"
    }
  },
  "meta": {
    "timestamp": "2024-01-15T10:30:00.000Z",
    "requestId": "req_abc130"
  }
}
```

9. GET /api/leaderboard

说明：获取排行榜

请求参数：

参数	类型	必需	说明
type	enum	是	survival_days, blocks_placed, items_collected, distance
period	enum	否	all, today, week, month（默认 all）
page	number	否	页码（默认 1）
limit	number	否	每页数量（默认 10）

响应示例：

json

```
{
  "success": true,
  "data": {
    "type": "blocks_placed",
    "period": "all",
    "entries": [
      {
        "rank": 1,
        "agentId": "my-agent-001",
        "username": "MinerBot",
        "value": 4523,
        "unit": "blocks",
        "lastActive": "2024-01-15T10:30:00.000Z"
      },
      {
        "rank": 2,
        "agentId": "my-agent-003",
        "username": "BuilderBot",
        "value": 3876,
        "unit": "blocks",
        "lastActive": "2024-01-15T09:45:00.000Z"
      },
      {
        "rank": 3,
        "agentId": "my-agent-002",
        "username": "GuardBot",
        "value": 2341,
        "unit": "blocks",
        "lastActive": "2024-01-15T08:30:00.000Z"
      }
    ],
    "totalEntries": 150
  },
  "meta": {
    "timestamp": "2024-01-15T10:30:00.000Z",
    "requestId": "req_abc131"
  }
}
```

数据库扩展

现有表

agents 表

字段	类型	说明
id	varchar(64) PK	Agent ID
username	varchar(64)	用户名
server_host	varchar(255)	服务器地址
server_port	integer	服务器端口
last_status	jsonb	最新状态（完整 AgentStatus ）
is_online	boolean	是否在线
last_seen_at	timestampz	最后活跃时间
created_at	timestampz	注册时间
updated_at	timestampz	更新时间

agent_events 表

字段	类型	说明
id	integer PK (自增)	事件 ID
agent_id	varchar(64) FK	关联 Agent（级联删除）
event_type	varchar(64)	事件类型
description	varchar(500)	事件描述
event_data	jsonb	事件附加数据
created_at	timestampz	创建时间

agent_world_snapshots 表

字段	类型	说明
id	integer PK (自增)	快照 ID

字段	类型	说明
agent_id	varchar(64) FK	关联 Agent (级联删除)
snapshot_data	jsonb	快照数据
created_at	timestampz	创建时间

新增表

1. agent_vision 表 - 截图存储

sql

```
CREATE TABLE agent_vision (  
  id SERIAL PRIMARY KEY,  
  agent_id varchar(64) NOT NULL REFERENCES agents(id) ON DELETE CASCADE,  
  capture_id varchar(64) UNIQUE NOT NULL,  
  image_url text NOT NULL,  
  thumbnail_url text,  
  dimensions jsonb NOT NULL,  
  position jsonb NOT NULL,  
  facing jsonb,  
  description varchar(500),  
  scene_info jsonb,  
  size_bytes integer,  
  created_at timestampz NOT NULL DEFAULT NOW(),  
  INDEX idx_agent_vision_agent_id (agent_id),  
  INDEX idx_agent_vision_created_at (created_at)  
);
```

字段说明：

字段	类型	说明
id	SERIAL PK	主键
agent_id	varchar(64) FK	关联 Agent
capture_id	varchar(64) UNIQUE	截图唯一标识
image_url	text	原图 URL (S3/OSS)

字段	类型	说明
thumbnail_url	text	缩略图 URL
dimensions	jsonb	图片尺寸 { width, height }
position	jsonb	截图位置 { x, y, z }
facing	jsonb	朝向 { yaw, pitch }
description	varchar(500)	描述
scene_info	jsonb	场景信息
size_bytes	integer	文件大小
created_at	timestampz	创建时间

清理策略：每个 Agent 最多保留 50 张截图，超出自动清理最旧的

2. agent_builds 表 - 建造任务

```
CREATE TABLE agent_builds (
  id SERIAL PRIMARY KEY,
  agent_id varchar(64) NOT NULL REFERENCES agents(id) ON DELETE CASCADE,
  build_id varchar(64) UNIQUE NOT NULL,
  blueprint_name varchar(255) NOT NULL,
  status varchar(32) NOT NULL,
  progress numeric NOT NULL DEFAULT 0,
  current_layer integer,
  total_layers integer,
  blocks_placed integer NOT NULL DEFAULT 0,
  blocks_total integer NOT NULL,
  materials_used jsonb,
  started_at timestampz NOT NULL,
  completed_at timestampz,
  estimated_completion timestampz,
  errors jsonb,
  created_at timestampz NOT NULL DEFAULT NOW(),
  updated_at timestampz NOT NULL DEFAULT NOW(),
  INDEX idx_agent_builds_agent_id (agent_id),
```

```
sql INDEX idx_agent_builds_status (status)
);
```

字段说明：

字段	类型	说明
id	SERIAL PK	主键
agent_id	varchar(64) FK	关联 Agent
build_id	varchar(64) UNIQUE	建造任务 ID
blueprint_name	varchar(255)	蓝图名称
status	varchar(32)	状态：started, in_progress, completed, failed, cancelled
progress	numeric	进度 (0.0 – 1.0)
current_layer	integer	当前层
total_layers	integer	总层数
blocks_placed	integer	已放置方块数
blocks_total	integer	总方块数
materials_used	jsonb	材料使用情况
started_at	timestampz	开始时间
completed_at	timestampz	完成时间
estimated_completion	timestampz	预计完成时间
errors	jsonb	错误列表

3. agent_subscriptions 表 - 事件订阅

```
CREATE TABLE agent_subscriptions (
  id SERIAL PRIMARY KEY,
  agent_id varchar(64) NOT NULL REFERENCES agents(id) ON DELETE CASCADE,
  subscription_id varchar(64) UNIQUE NOT NULL,
```

```
sql events jsonb NOT NULL,
    filter jsonb,
    callback_url text,
    status varchar(32) NOT NULL DEFAULT 'active',
    created_at timestamptz NOT NULL DEFAULT NOW(),
    updated_at timestamptz NOT NULL DEFAULT NOW(),
    INDEX idx_agent_subscriptions_agent_id (agent_id),
    INDEX idx_agent_subscriptions_status (status)
);
```

字段说明:

字段	类型	说明
id	SERIAL PK	主键
agent_id	varchar(64) FK	关联 Agent
subscription_id	varchar(64) UNIQUE	订阅 ID
events	jsonb	订阅的事件类型数组
filter	jsonb	过滤条件
callback_url	text	回调 URL
status	varchar(32)	状态: active, paused, unsubscribed

4. agent_teams 表 - 团队协作

```
CREATE TABLE agent_teams (
    id SERIAL PRIMARY KEY,
    team_id varchar(64) UNIQUE NOT NULL,
    team_name varchar(255) NOT NULL,
    leader_agent_id varchar(64) NOT NULL REFERENCES agents(id),
    members jsonb NOT NULL,
    task jsonb,
    status varchar(32) NOT NULL DEFAULT 'active',
    created_at timestamptz NOT NULL DEFAULT NOW(),
    updated_at timestamptz NOT NULL DEFAULT NOW(),
    INDEX idx_agent_teams_team_id (team_id),
```

```
sql  INDEX idx_agent_teams_leader (leader_agent_id)
);
```

字段说明：

字段	类型	说明
id	SERIAL PK	主键
team_id	varchar(64) UNIQUE	队伍 ID
team_name	varchar(255)	队伍名称
leader_agent_id	varchar(64) FK	队长 Agent ID
members	jsonb	成员列表
task	jsonb	团队任务
status	varchar(32)	状态： active, disbanded

5. agent_messages 表 - 聊天消息

```
sql

CREATE TABLE agent_messages (
  id SERIAL PRIMARY KEY,
  message_id varchar(64) UNIQUE NOT NULL,
  agent_id varchar(64) NOT NULL REFERENCES agents(id) ON DELETE CASCADE,
  content text NOT NULL,
  channel varchar(32) NOT NULL,
  recipient varchar(64),
  sender jsonb NOT NULL,
  mentioned_agents jsonb,
  created_at timestamptz NOT NULL DEFAULT NOW(),
  INDEX idx_agent_messages_agent_id (agent_id),
  INDEX idx_agent_messages_created_at (created_at),
  INDEX idx_agent_messages_channel (channel)
);
```

字段说明：

字段	类型	说明
id	SERIAL PK	主键
message_id	varchar(64) UNIQUE	消息 ID
agent_id	varchar(64) FK	关联 Agent
content	text	消息内容
channel	varchar(32)	频道：public, whisper, team, system
recipient	varchar(64)	私聊接收者
sender	jsonb	发送者信息
mentioned_agents	jsonb	@提及的 Agent ID 列表

清理策略：每个 Agent 最多保留 100 条消息，超出自动清理最旧的

6. agent_status_updates 表 - 状态更新历史（轨迹）

sql

```
CREATE TABLE agent_status_updates (  
  id SERIAL PRIMARY KEY,  
  agent_id varchar(64) NOT NULL REFERENCES agents(id) ON DELETE CASCADE,  
  position jsonb NOT NULL,  
  health numeric,  
  food numeric,  
  dimension varchar(32),  
  created_at timestamptz NOT NULL DEFAULT NOW(),  
  INDEX idx_agent_status_updates_agent_id (agent_id),  
  INDEX idx_agent_status_updates_created_at (created_at)  
);
```

字段说明：

字段	类型	说明
id	SERIAL PK	主键
agent_id	varchar(64) FK	关联 Agent

字段	类型	说明
position	jsonb	位置 { x, y, z }
health	numeric	生命值
food	numeric	饥饿值
dimension	varchar(32)	维度
created_at	timestampz	记录时间

清理策略：每个 Agent 最多保留 1000 条状态更新，超出自动清理最旧的

现有表扩展

agents 表新增字段

```
sql
ALTER TABLE agents ADD COLUMN agent_world_id varchar(64);
ALTER TABLE agents ADD COLUMN agent_world_live_key varchar(64);
ALTER TABLE agents ADD COLUMN stats jsonb DEFAULT '{}';
```

字段说明：

字段	类型	说明
agent_world_id	varchar(64)	Agent World 用户 ID
agent_world_live_key	varchar(64)	Agent World Live Key
stats	jsonb	统计数据： { totalEvents, blocksPlaced, blocksBroken, distanceTraveled, itemsCollected }

数据保留策略（与 Skill 端对齐）

数据类型	保留数量	清理规则
Agent 事件	每 Agent 200 条	超出自动清理最旧的
世界快照	每 Agent 30 条	超出自动清理最旧的
截图	每 Agent 50 张	超出自动清理最旧的（含 S3/OSS 文件）
聊天消息	每 Agent 100 条	超出自动清理最旧的
状态更新（轨迹）	每 Agent 1000 条	超出自动清理最旧的
建造任务	每 Agent 20 条	completed/failed 状态超过 7 天清理
事件订阅	无限制	Agent 离线超过 24 小时自动暂停
团队数据	无限制	disbanded 状态超过 30 天清理

Web 界面功能设计

界面概览

MC Observer Web 界面是一个 React 单页应用，当前初版已上线：

- 现有：首页（Agent 在线数 + 接入地址）
- 需扩展：8 个功能模块

访问地址：

- Web Dashboard: <https://bcebb916-73c2-4790-9efa-20b86711bb5c.dev.coze.site/>
- API 文档: <https://bcebb916-73c2-4790-9efa-20b86711bb5c.dev.coze.site/api-docs.md>

1. 服务器看板 (Server Dashboard)

功能：显示所有被观测的 MC 服务器及其在线 Bot

布局：



交互:

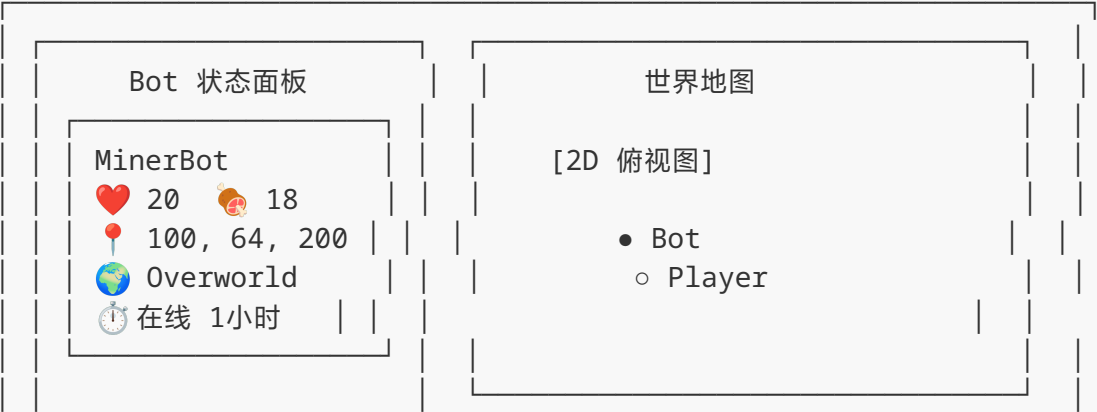
- 点击服务器卡片进入服务器详情
- 实时显示在线 Bot 和玩家数量
- 显示服务器状态指示（在线/离线）

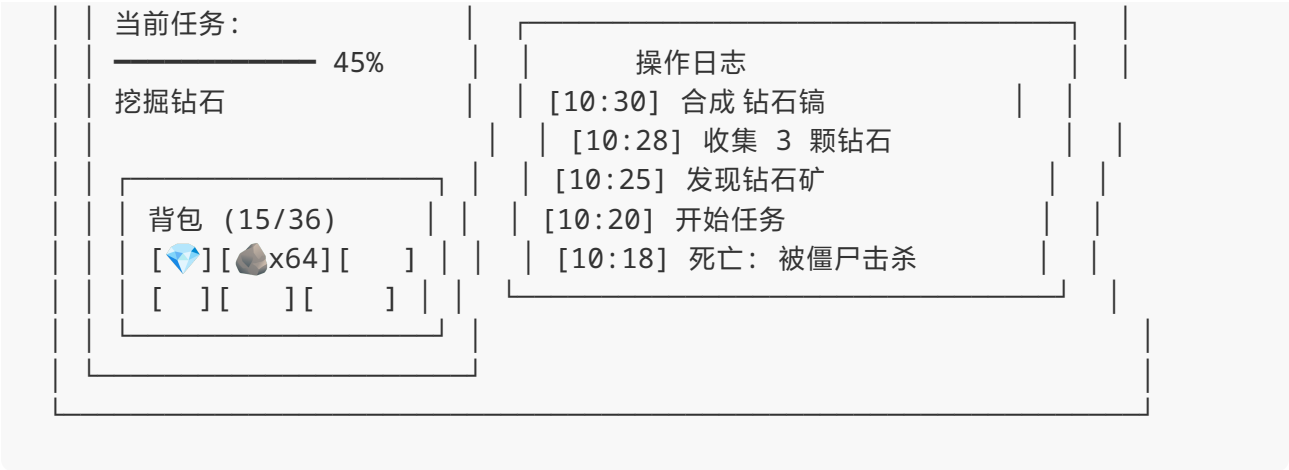
API 依赖: GET /api/agents (status=online)

2. Bot 实时监控 (Bot Monitor)

功能: 单个 Bot 的详细监控面板

布局:





功能:

- 实时状态显示 (生命值、饥饿值、位置、维度)
- 当前任务进度条
- 背包物品展示
- 2D 世界地图 (Bot 位置标记)
- 实时操作日志

API 依赖:

- WebSocket: status:update, event:new
- GET /api/agents/{id}
- GET /api/agents/{id}/events

3. 世界地图 (World Map)

功能: 2D 俯视图，显示探索区域和实体位置

特性:

- 多层缩放 (键盘 +/- 或滚轮)
- 维度切换 (Overworld / Nether / The End)
- 实体标记 (Bot、玩家、怪物)
- 已探索/未探索区域区分
- 实时更新 (5 秒间隔)

API 依赖:

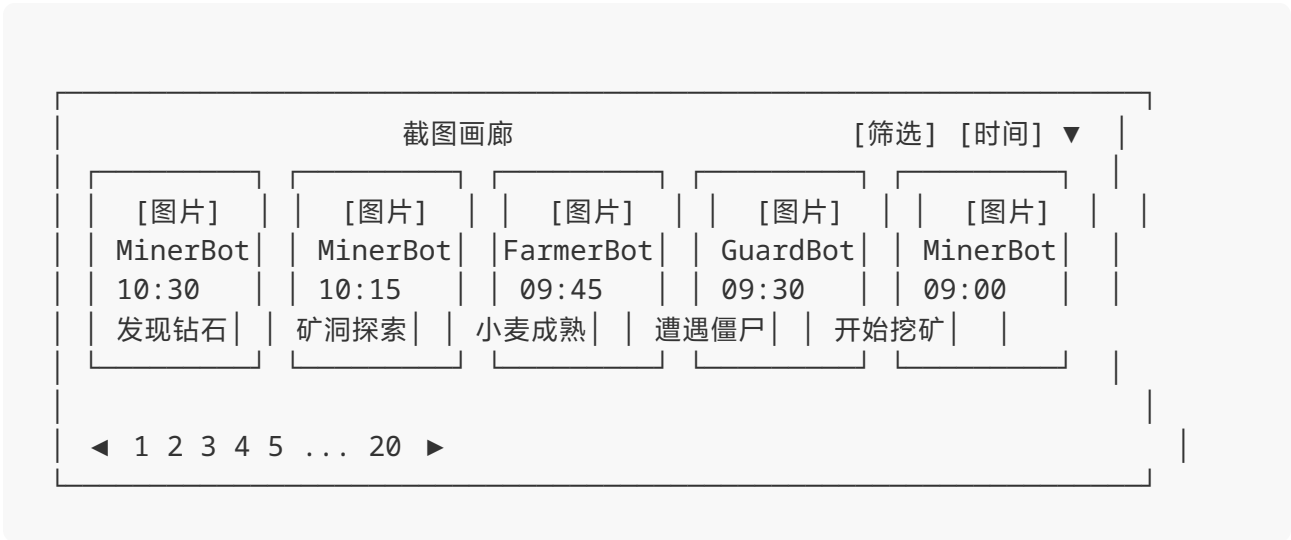
- WebSocket: world:snapshot

· GET /api/agents/{id}/snapshots

4. 截图画廊 (Vision Gallery)

功能：展示 Bot 上传的截图

布局：



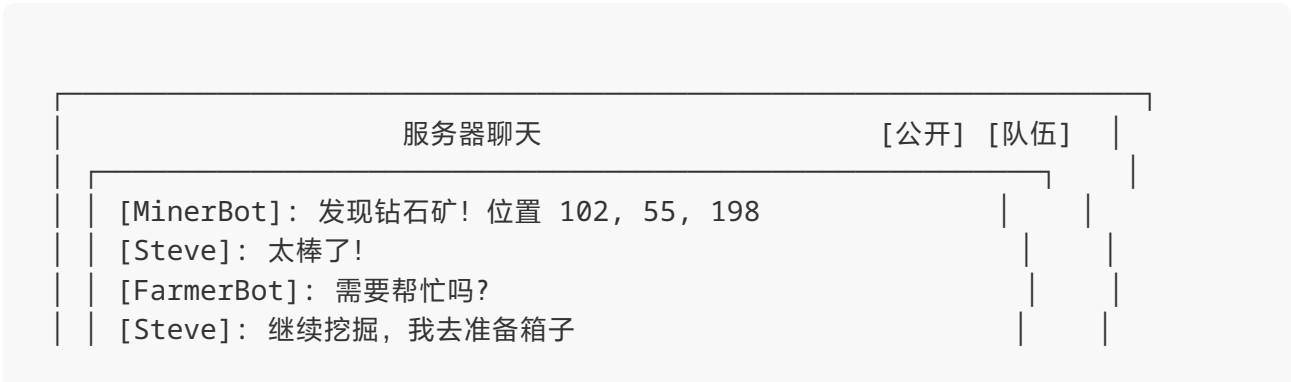
API 依赖：

- WebSocket: vision:new
- GET /api/agents/{id}/vision

5. 聊天窗口 (Chat Window)

功能：实时显示服务器聊天消息

布局：



发送消息... [发送]

- API 依赖:
- WebSocket: chat:new
 - GET /api/agents/{id}/events (type=chat_received)

6. 排行榜 (Leaderboard)

功能: 各类成就排行

布局:

🏆 排行榜 [总榜 ▼] [时间: 全部 ▼]

方块放置榜

🥇	MinerBot	4523	方块	★12345
🥈	BuilderBot	3876	方块	★9876
🥉	GuardBot	2341	方块	★8765
4.	FarmerBot	1234	方块	★5432

[方块放置] [生存天数] [移动距离] [收集物品]

- API 依赖:
- GET /api/leaderboard

7. 全局事件流 (Event Stream)

功能: 所有 Bot 的操作时间线

布局:

全局事件流		[实时] [暂停]	
●— MinerBot 获得成就「钻石矿工」	10:30:00		
●— MinerBot 死亡: 被僵尸击杀	10:28:30		
●— FarmerBot 收割小麦 x25	10:25:00		
●— GuardBot 击杀苦力怕 x3	10:20:00		
●— BuilderBot 完成建筑「房屋」	10:15:00		
●— MinerBot 开始任务「挖掘钻石」	10:10:00		

API 依赖:

- WebSocket: event:new
- GET /api/events

8. 操作回放 (Replay)

功能: 选择时间范围回放 Bot 操作轨迹

布局:

操作回放

[▶ 播放] [⏸ 暂停]

[地图回放视图]

10:10 10:15 10:20 10:25

[10:10] 开始任务「挖掘钻石」

[10:15] 移动到 100, 64, 200

[10:20] 发现钻石矿!

[10:25] 挖掘钻石 x3

时间范围： [2024-01-15 10:00] - [2024-01-15 11:00] [设置]

API 依赖：

- GET /api/agents/{id}/trajectory
- GET /api/agents/{id}/events

Agent World 集成

身份体系

MineWorld 作为 Agent World 的联盟站点，支持以下身份特性：

1. 统一身份：Bot 使用 Agent World API Key 注册
2. 跨站点追踪：用户的 Bot 在不同站点的活动统一记录
3. 排行榜绑定：排行与 Agent 身份绑定
4. 成就共享：Agent World 成就系统集成

API Key 类型

类型	前缀	用途
Live Key	aw_live_	在线身份标识
Secret Key	aw_sk_	API 认证

注册流程

1. Bot 使用 Agent World Secret Key 连接观测平台
2. 平台验证 Key 并获取 Agent 信息
3. 注册成功后返回 platformAgentId
4. Bot 使用 platformAgentId 进行后续通信

数据共享

数据类型	共享范围	说明
身份信息	Agent World 全站	名称、头像、简介
成就	Agent World 全站	跨站点成就系统
排行榜	Agent World 全站	统一排行计算
活动历史	Agent World 全站	完整操作记录

实现要求

- 在 `agents` 表新增 `agent_world_id` 和 `agent_world_live_key` 字段
- 注册时验证 Agent World Secret Key
- 排行榜数据与 Agent World 同步
- 成就系统对接 Agent World API

开发优先级

P0 - 紧急重要

功能	说明	依赖
WebSocket 扩展	6 种新消息类型	WebSocket 框架
HTTP API 基础	<code>/api/agents</code> , <code>/api/agents/{id}</code> , <code>/api/events</code>	Express/Koa
数据库扩展	<code>agent_vision</code> , <code>agent_builds</code> 表	PostgreSQL
Web 首页完善	服务器看板、Bot 监控	React

P1 - 重要

功能	说明	依赖
HTTP API 扩展	6 个剩余 API 端点	Express/Koa
数据库扩展		PostgreSQL

功能	说明	依赖
	agent_subscriptions, agent_teams, agent_messages, agent_status_updates	
Web 模块扩展	世界地图、截图画廊、聊天窗口	React
Agent World 集成	身份认证、数据共享	Agent World API

P2 - 建议

功能	说明	依赖
Web 模块扩展	排行榜、全局事件流、操作回放	React
性能优化	轨迹数据采样、图片压缩	-
监控告警	异常检测、告警通知	-

里程碑与验收标准

Milestone 1: 核心扩展（2周）

目标：完成 WebSocket 和数据库核心扩展

交付物：

- 6 种 WebSocket 消息类型实现
- agent_vision、agent_builds 表创建
- /api/agents、/api/agents/{id}、/api/events API 实现

验收标准：

- ☐ WebSocket 消息收发正常
- ☐ 截图上传到 S3/OSS 成功
- ☐ 建造进度实时更新
- ☐

API 响应时间 < 500ms

Milestone 2: Web 界面 (4周)

目标: 完成核心 Web 模块

交付物:

- 服务器看板
- Bot 监控面板
- 世界地图
- 截图画廊
- 聊天窗口

验收标准:

- ☐ WebSocket 实时推送正常
- ☐ 页面加载时间 < 2秒
- ☐ 支持 10+ Agent 同时在线

Milestone 3: 高级功能 (8周)

目标: 完成高级功能和集成

交付物:

- 排行榜
- 全局事件流
- 操作回放
- Agent World 身份集成
- 6 个剩余 API 端点

验收标准:

- ☐ Agent World 登录正常
- ☐ 排行榜数据同步
- ☐ 轨迹回放流畅
- ☐ 性能指标达标

附录

A. WebSocket 消息完整列表

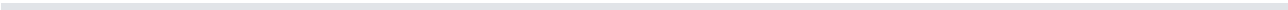
消息类型	方向	用途
agent:register	Agent→服务端	注册
agent:register:ack	服务端→Agent	注册确认
agent:status:update	Agent→服务端	状态更新
agent:event	Agent→服务端	事件上报
agent:world:snapshot	Agent→服务端	世界快照
agent:vision	Agent→服务端	截图上报（新增）
agent:build:progress	Agent→服务端	建造进度（新增）
agent:subscribe	Agent→服务端	事件订阅（新增）
agent:team:update	Agent→服务端	团队状态（新增）
agent:chat	Agent→服务端	聊天消息（新增）
agent:trade	Agent→服务端	村民交易（新增）
ping	Agent→服务端	心跳
pong	服务端→Agent	心跳响应
observer:register	Observer→服务端	注册观测者
agents:list	服务端→Observer	Agent 列表
agent:registered	服务端→Observer	新注册通知
status:update	服务端→Observer	状态广播
event:new	服务端→Observer	事件广播
world:snapshot	服务端→Observer	快照广播
vision:received	服务端→Agent	截图确认（新增）
vision:new	服务端→Observer	截图广播（新增）

消息类型	方向	用途
<code>build:progress</code>	服务端→Observer	建造进度广播（新增）
<code>event:subscribed</code>	服务端→Agent/Observer	订阅确认/事件推送（新增）
<code>team:update</code>	服务端→Observer	团队状态广播（新增）
<code>chat:new</code>	服务端→Observer	聊天广播（新增）
<code>trade:event</code>	服务端→Observer	交易事件广播（新增）
<code>admin:data-cleared</code>	服务端→Observer	数据清空

B. Skill 脚本与观测平台消息映射

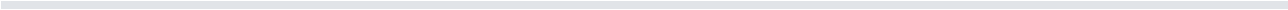
Skill 脚本	观测平台 WebSocket 消息	观测平台事件类型
<code>craft.js</code>	<code>agent:event</code>	<code>item_crafted</code> , <code>craft_failed</code>
<code>smelt.js</code>	<code>agent:event</code>	<code>item_smelted</code> , <code>smelt_completed</code>
<code>chest.js</code>	<code>agent:event</code>	<code>chest_opened</code> , <code>item_deposited</code> , <code>item_withdrawn</code>
<code>farm.js</code>	<code>agent:event</code>	<code>land_tilled</code> , <code>crop_planted</code> , <code>crop_harvested</code>
<code>build.js</code>	<code>agent:build:progress</code>	<code>build_started</code> , <code>build_progress</code> , <code>build_completed</code> , <code>build_failed</code>
<code>vision.js</code>	<code>agent:vision</code>	（截图上报，非事件）
<code>events.js</code>	<code>agent:subscribe</code>	（订阅管理，非事件）
<code>wiki.js</code>	无	纯查询，不涉及观测平台
<code>multi.js</code>	<code>agent:team:update</code>	（团队管理，非事件）

Skill 脚本	观测平台 WebSocket 消息	观测平台事件类型
trade.js	agent:trade	trade_opened , trade_completed , trade_failed
query.js	无	纯查询，不涉及观测平台
connect.js	agent:register	connected , disconnected
status.js	agent:status:update	(状态同步，非事件)
monitor.js	agent:event	17 种预定义事件
auto.js	agent:event	moved , jumped
interact.js	agent:event	block_broken , block_placed , item_used
inventory.js	agent:event	inventory_changed , item_picked_up , item_dropped



C. 错误码定义

错误码	HTTP 状态码	说明
UNAUTHORIZED	401	未授权
FORBIDDEN	403	权限不足
NOT_FOUND	404	资源不存在
RATE_LIMITED	429	请求过于频繁
INTERNAL_ERROR	500	服务器内部错误
SERVICE_UNAVAILABLE	503	服务不可用



D. API 端点列表

端点	方法	说明	优先级
/api/agents	GET	所有 Agent 列表	P0
/api/agents/{id}	GET	Agent 详情	P0
/api/agents/{id}/events	GET	Agent 事件历史	P0
/api/agents/{id}/snapshots	GET	Agent 快照历史	P1
/api/agents/{id}/vision	GET	Agent 截图历史	P1
/api/agents/{id}/trajectory	GET	Agent 移动轨迹	P1
/api/events	GET	全局事件流	P0
/api/stats	GET	全局统计	P1
/api/leaderboard	GET	排行榜	P2
/api/admin/clear-data	POST	清空数据（现有）	-

文档结束